

Nome: Guilherme de Moraes China Costa

Data: 25/10/2025

Título: Engenharia de Software: Clean Code e Boas Práticas - TP1

Exercício 1: Explicar por que a qualidade do código é importante

A qualidade do código é crucial porque determina a facilidade de manutenção, leitura e extensão de um software. Um código de baixa qualidade gera "dívida técnica", tornando cada alteração futura mais lenta, cara e propensa a erros.

Problemas Estruturais Identificados:

1. **Listas Paralelas (Baixa Coesão):** O código usa List contas e List valores. Os dados de uma mesma entidade (a conta) estão espalhados por duas estruturas. Isso é extremamente frágil: se um item for adicionado a uma lista mas não à outra, todos os dados do sistema ficam corrompidos.
2. **Ausência de Modelo de Domínio:** Não existe uma classe Conta. O sistema é forçado a usar tipos primitivos (String, double) para representar um conceito complexo, o que esconde a intenção do código.
3. **Tipagem Bruta (Raw Types):** As listas são declaradas como List em vez de List<String> ou List<Double>. Isso elimina a segurança de tipos em tempo de compilação, podendo causar ClassCastException em tempo de execução.

Impacto na Qualidade e Manutenção:

A manutenção desse código é insustentável. Se precisarmos adicionar um novo atributo (ex: "tipo da conta"), seríamos forçados a criar uma terceira lista paralela, aumentando a complexidade exponencialmente. O código não é robusto e depende da sincronia manual dos índices das listas, o que é uma receita para bugs.

Exercício 2: Enumerar os quatro objetivos de um código de alta qualidade

O código apresentado falha em atingir os quatro objetivos principais da seguinte forma:

1. **Legibilidade:** O código usa aninhamento excessivo (if-else dentro de if-else), criando uma "estrutura de seta" (arrow code) que é difícil de acompanhar mentalmente. Seria mais legível usar "Guard Clauses" (cláusulas de guarda) para tratar os erros primeiro.
2. **Redução de Duplicação:** A lógica de validação (nulo, tamanho) está misturada com a lógica de processamento (o System.out.println). Além disso, a ação de "reportar um resultado" (System.out.println) está duplicada em três locais diferentes.
3. **Facilidade de Modificação:** O código é rígido. Adicionar uma nova regra de validação exigiria outro if aninhado, piorando ainda mais a legibilidade. A lógica de validação não está isolada.
4. **Eficácia na Comunicação:** O código comunica mal sua intenção. O nome processar é vago, e o número 10 é um "número mágico" sem contexto (o que ele representa?).

Exercício 3: Garantir a qualidade do código estrategicamente

Problemas Estruturais e Impacto:

Esta classe sofre dos mesmos problemas do Exercício 1:

Listas Paralelas: O uso de produtos, preços e quantidades é extremamente frágil. Se um item for removido de uma lista mas não das outras, o cálculo do total quebrará ou gerará valores incorretos.

Baixa Coesão (Violação do SRP): O método `calcularTotal()` mistura duas responsabilidades: ele calcula o total e apresenta o total (`System.out.println`). Isso acopla a lógica de negócios diretamente ao console, impedindo que esse cálculo seja reutilizado em uma interface gráfica ou API.

A manutenibilidade é baixa, pois qualquer alteração (ex: adicionar desconto) exige a modificação de três listas. A extensibilidade é quase nula pelo mesmo motivo.

Solução Proposta:

A solução decompõe o problema, criando uma classe coesa para o item e separando o cálculo da apresentação.

Classe ItemCarrinho (Camada de Modelo)

Esta classe encapsula os dados que pertencem juntos.

```
Java
/**
 * Representa um item individual dentro do carrinho de compras.
 * Encapsula os dados do produto e a lógica de subtotal.
 */
public class ItemCarrinho {
    private String nome;
    private double precoUnitario;
    private int quantidade;

    public ItemCarrinho(String nome, double precoUnitario, int quantidade) {
        // Validações (ex: preço/quantidade > 0) podem ser adicionadas aqui.
        this.nome = nome;
        this.precoUnitario = precoUnitario;
        this.quantidade = quantidade;
    }

    /**
     * Calcula o subtotal para este item (preço * quantidade).
     * @return o subtotal do item.
     */
    public double getSubtotal() {
        return this.precoUnitario * this.quantidade;
    }

    // Getters
    public String getNome() { return nome; }
    public double getPrecoUnitario() { return precoUnitario; }
    public int getQuantidade() { return quantidade; }
}
```

Classe CarrinhoDeCompras (Camada de Serviço/Agregação)

Esta classe gerencia a coleção de itens e a lógica de cálculo total, fornecendo uma API limpa.

Java

```
import java.util.ArrayList;
import java.util.List;
```

```
/**
 * Gerencia uma coleção de itens de compra e calcula o valor total.
 * A lógica de cálculo é desacoplada da exibição.
 */
public class CarrinhoDeCompras {

    // Usamos uma única lista de um objeto coeso.
    private final List<ItemCarrinho> itens;

    public CarrinhoDeCompras() {
        this.itens = new ArrayList<>();
    }

    /**
     * Adiciona um item ao carrinho.
     * Esta função é pequena e tem propósito claro.
     */
    public void adicionarItem(String nome, double preco, int quantidade) {
        // A lógica de criação do item fica encapsulada
        ItemCarrinho item = new ItemCarrinho(nome, preco, quantidade);
        this.itens.add(item);
    }

    /**
     * Calcula o valor total da compra somando o subtotal de todos os itens.
     * Esta função tem um propósito único e RETORNA o resultado.
     *
     * @return o valor total da compra.
     */
    public double calcularTotal() {
        double total = 0.0;
        for (ItemCarrinho item : this.itens) {
            total += item.getSubtotal();
        }
        return total; // Desacoplado do System.out
    }

    public List<ItemCarrinho> getItens() {
        return new ArrayList<>(this.itens); // Retorna cópia
    }
}
```

Esta solução é modular, testável e reutilizável, pois `calcularTotal()` pode ser usado por qualquer interface (console, web, etc.).

Exercício 4: Dividir um problema em subproblemas com camadas de abstração

Análise de Responsabilidades:

O SistemaBiblioteca viola o Princípio da Responsabilidade Única (SRP). Ele gerencia duas coisas:

1. **O Acervo:** A coleção de livros.
2. **O Estado do Livro:** Controla se um livro específico está disponível.

A disponibilidade é um atributo do livro, não da biblioteca. A classe SistemaBiblioteca não deveria gerenciar o estado interno de outra entidade através de listas paralelas.

Solução Proposta:

Dividimos o problema em duas classes, cada uma com sua responsabilidade:

Classe Livro

Representa o modelo de dados. Ele gerencia seu próprio estado de disponibilidade.

```
Java
/**
 * Representa um único livro, gerenciando seu próprio título
 * e estado de disponibilidade.
 */
public class Livro {
    private String titulo;
    private boolean disponivel;

    public Livro(String titulo) {
        this.titulo = titulo;
        this.disponivel = true; // Todo livro começa disponível
    }

    public String getTitulo() {
        return titulo;
    }

    public boolean isDisponivel() {
        return disponivel;
    }

    /**
     * Tenta emprestar o livro. Retorna true se bem-sucedido.
     */
    public boolean emprestar() {
        if (this.disponivel) {
            this.disponivel = false;
            return true;
        }
        return false; // Já estava emprestado
    }

    /**
     * Devolve o livro, tornando-o disponível.
     */
}
```

```
        public void devolver() {
            this.disponivel = true;
        }
    }
}
```

Classe Biblioteca

Representa o serviço (acervo). Ela gerencia a coleção de objetos Livro.

```
Java
import java.util.ArrayList;
import java.util.List;

/**
 * Gerencia o acervo de livros e orchestra as operações de empréstimo
 * e devolução, delegando a lógica de estado para o objeto Livro.
 */
public class Biblioteca {

    private List<Livro> acervo;

    public Biblioteca() {
        this.acervo = new ArrayList<>();
    }

    public void adicionarLivro(String titulo) {
        this.acervo.add(new Livro(titulo));
    }

    // Método auxiliar privado para abstrair a busca
    private Livro buscarLivroPorTitulo(String titulo) {
        for (Livro livro : acervo) {
            if (livro.getTitulo().equals(titulo)) {
                return livro;
            }
        }
        return null;
    }

    public void emprestarLivro(String titulo) {
        Livro livro = buscarLivroPorTitulo(titulo);

        if (livro != null) {
            if (livro.emprestar()) { // Delega a lógica para o objeto Livro
                System.out.println("Livro emprestado: " + titulo);
            } else {
                System.out.println("Livro já está emprestado.");
            }
        } else {
            System.out.println("Livro não encontrado no acervo.");
        }
    }

    public void devolverLivro(String titulo) {
        Livro livro = buscarLivroPorTitulo(titulo);

        if (livro != null) {
            livro.devolver(); // Delega a lógica
            System.out.println("Livro devolvido: " + titulo);
        }
    }
}
```

```

    } else {
        System.out.println("Livro não encontrado.");
    }
}
}
}

```

Exercício 5: Dividir códigos em diferentes camadas de abstração

O SistemaBancario original mistura todas as camadas: Modelo (dados em listas paralelas), Serviço (lógica de transferência) e Interface (saída com System.out.println).

A funcionalidade foi separada nas três camadas distintas:

1. Camada de Modelo (Model)

Representa a entidade de dados (Conta). Ela contém os dados e as regras de negócio que se aplicam a ela mesma (ex: não permitir saldo negativo).

```

Java
/**
 * Camada de Modelo. Representa uma única conta bancária.
 * Protege seu próprio estado (regras de negócio).
 */
public class Conta {
    private String titular;
    private double saldo;

    public Conta(String titular, double saldoInicial) {
        this.titular = titular;
        this.saldo = saldoInicial;
    }

    public String getTitular() { return titular; }
    public double getSaldo() { return saldo; }

    public void depositar(double valor) {
        if (valor > 0) {
            this.saldo += valor;
        }
    }

    /**
     * @return true se o saque foi bem-sucedido, false caso contrário.
     */
    public boolean sacar(double valor) {
        if (valor <= 0 || valor > this.saldo) {
            return false; // Saldo insuficiente ou valor inválido
        }
        this.saldo -= valor;
        return true;
    }
}

```

2. Camada de Serviço (Service)

Orquestra as operações entre as entidades. Ela implementa a lógica de negócio que envolve múltiplas Contas (como a transferência), mas não lida com a interface do usuário.

```
Java
import java.util.ArrayList;
import java.util.List;

/**
 * Camada de Serviço. Gerencia a coleção de contas e orquestra
 * operações complexas entre elas (ex: transferência).
 */
public class BancoService {
    private List<Conta> contas;

    public BancoService() {
        this.contas = new ArrayList<>();
    }

    public void criarConta(String titular, double saldoInicial) {
        this.contas.add(new Conta(titular, saldoInicial));
    }

    private Conta buscarContaPorTitular(String titular) {
        for (Conta conta : contas) {
            if (conta.getTitular().equals(titular)) {
                return conta;
            }
        }
        return null;
    }

    /**
     * Orquestra a transferência.
     * @return true se a transferência foi bem-sucedida.
     */
    public boolean transferir(String titularOrigem, String titularDestino, double valor) {
        Conta origem = buscarContaPorTitular(titularOrigem);
        Conta destino = buscarContaPorTitular(titularDestino);

        if (origem == null || destino == null) {
            return false; // Contas não encontradas
        }

        // Delega a lógica de saque/depósito para o Modelo
        if (origem.sacar(valor)) {
            destino.depositar(valor);
            return true;
        }

        return false; // Saldo insuficiente
    }
}
```

3. Camada de Interface (View/Controller)

Lida com a interação do usuário (console, neste caso) e traduz as ações em chamadas para a Camada de Serviço.

```
Java
/**
 * Camada de Interface (Apresentação).
 * Responsável por interagir com o usuário e usar a camada de serviço.
 */
public class TerminalBancario {

    public static void main(String[] args) {
        // A Interface "usa" o Serviço
        BancoService meuBanco = new BancoService();
        meuBanco.criarConta("Ana", 1000.0);
        meuBanco.criarConta("Beto", 500.0);

        // Ação do usuário: Transferir
        boolean sucesso = meuBanco.transferir("Ana", "Beto", 200.0);

        // A Interface apresenta o resultado
        if (sucesso) {
            System.out.println("Transferência realizada com sucesso.");
        } else {
            System.out.println("Erro na transferência.");
        }
    }
}
```

Exercício 6: Escrever funções pequenas e coesas

O método gerarRelatorio original viola o Princípio da Responsabilidade Única (SRP), pois faz três coisas: imprime o cabeçalho, o corpo e o rodapé, tudo em um só lugar.

A solução divide a funcionalidade em métodos menores e com propósitos claros, onde gerarRelatorio atua como um "orquestrador".

```
Java
import java.util.List;

public class RelatorioFinanceiro {

    /**
     * Orquestra a geração do relatório, delegando as partes
     * para métodos especializados. (Genéricos foram adicionados).
     */
    public void gerarRelatorio(List<String> clientes, List<Double> saldos) {
        imprimirCabeçalho();
        imprimirCorpo(clientes, saldos);
        imprimirRodape();
    }

    /**
     * Imprime o cabeçalho padronizado do relatório.
     * (Responsabilidade Única: Formatar Cabeçalho)
     */
    private void imprimirCabeçalho() {
```



```

        System.out.println("=== Relatório Financeiro ===");
    }

    /**
     * Processa as listas e imprime o conteúdo principal.
     * (Responsabilidade Única: Formatar Corpo)
     */
    private void imprimirCorpo(List<String> clientes, List<Double> saldos) {
        for (int i = 0; i < clientes.size(); i++) {
            imprimirLinhaCliente(clientes.get(i), saldos.get(i));
        }
    }

    /**
     * Formata e imprime uma única linha de dados do cliente.
     */
    private void imprimirLinhaCliente(String cliente, double saldo) {
        System.out.println("Cliente: " + cliente + " - Saldo: R$ " + saldo);
    }

    /**
     * Imprime o rodapé padronizado do relatório.
     * (Responsabilidade Única: Formatar Rodapé)
     */
    private void imprimirRodape() {
        System.out.println("=====");
        System.out.println("Fim do Relatório");
    }
}

```

Exercício 7: Assegurar a clareza e robustez do código com contratos

Problemas de Estados Inválidos:

O código original permite que a ContaBancaria entre em estados inválidos, violando regras de negócio:

1. Depósito Negativo: depositar(-100.0) é permitido, funcionando como um saque.
2. Saque Negativo: sacar(-100.0) é permitido, funcionando como um depósito.
3. Saldo Negativo: sacar(100.0) (com saldo 50.0) é permitido, resultando em saldo de -50.0.

Solução com Validações (Contratos Explícitos):

Implementei validações (contratos) para garantir que o objeto nunca entre em um estado inválido. Os métodos agora falham ruidosamente (lançando exceções) se o contrato for violado.

Java

```

public class ContaBancaria {
    private String titular;
    private double saldo;

    /**
     * Construtor agora valida as entradas.
     * Contrato: Uma conta não pode ser criada com saldo negativo
     * ou titular inválido.
     */
}

```

```
public ContaBancaria(String titular, double saldo) {
    if (titular == null || titular.trim().isEmpty()) {
        throw new IllegalArgumentException("Titular não pode ser nulo ou vazio.");
    }
    if (saldo < 0) {
        throw new IllegalArgumentException("Saldo inicial não pode ser negativo.");
    }
    this.titular = titular;
    this.saldo = saldo;
}

/**
 * Contrato: O valor do depósito deve ser positivo.
 *
 * @param valor O valor a ser depositado (deve ser > 0).
 */
public void depositar(double valor) {
    if (valor <= 0) {
        throw new IllegalArgumentException("O valor do depósito deve ser positivo.");
    }
    this.saldo += valor;
}

/**
 * Contrato: O valor do saque deve ser positivo e
 * não pode ser maior que o saldo disponível.
 *
 * @param valor O valor a ser sacado (deve ser > 0 e <= saldo).
 */
public void sacar(double valor) {
    if (valor <= 0) {
        throw new IllegalArgumentException("O valor do saque deve ser positivo.");
    }
    if (valor > this.saldo) {
        // Sinaliza um estado de negócio inválido
        throw new IllegalStateException("Saldo insuficiente para este saque.");
    }
    this.saldo -= valor;
}

public double getSaldo() {
    return saldo;
}

public String getTitular() {
    return titular;
}
}
```

Exercício 8: Escrever comentários claros e sem redundâncias

O código original estava poluído com "comentários-ruído", que são redundantes e apenas repetem o que o código (que está bem nomeado) já diz (ex: `this.id = id; // Define o ID`).

O código foi limpo, removendo todos os comentários desnecessários. O código autoexplicativo é sempre preferível. Adicionei apenas um JavaDoc no topo da classe, que explica o propósito (o "porquê") da classe existir, que é um comentário útil e de alto nível.

```
Java
/**
 * Representa um Pedido de compra no sistema.
 * Esta classe armazena as informações básicas de um pedido,
 * como seu identificador, descrição e valor total.
 */
public class Pedido {

    private int id;
    private String descricao;
    private double valor;

    public Pedido(int id, String descricao, double valor) {
        this.id = id;
        this.descricao = descricao;
        this.valor = valor;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    // Métodos getDescricao() e getValor() também existiriam aqui.
}
```

Exercício 9: Escrever códigos organizados, legíveis e bem estruturados

O código original era funcional, mas ilegível devido à formatação inadequada (falta de indentação, chaves na mesma linha, falta de espaços e nomes de variáveis ruins).

O código foi corrigido aplicando formatação padrão Java: indentação correta, quebra de linhas para blocos (`{}`), espaçamento adequado e renomeação da variável `v` para `dado`, melhorando a comunicação.

```
Java
/**
 * Classe simples para demonstrar a formatação de código.
 */
public class Processador {

    /**
```

```
* Executa o processamento de um dado, imprimindo-o se não for nulo.
*
* @param dado O valor a ser processado.
*/
public void executar(String dado) {
    // A indentação correta e o espaçamento tornam o bloco claro
    if (dado != null) {
        System.out.println("Processando: " + dado);
    }
}
}
```

Exercício 10: Promover a robustez do código com tratamento claro de erros

O método processar não é robusto. Se for chamado com processar(null), o programa travará com uma NullPointerException (NPE) ao tentar executar null.toUpperCase().

A correção adiciona uma "Cláusula de Guarda" (Guard Clause) para validar a entrada antes de usá-la. Isso previne o crash e torna a falha visível e controlada.

```
Java
public class Servico {

    /**
     * Processa um dado, convertendo-o para maiúsculas.
     *
     * @param dado O dado a ser processado (não pode ser nulo).
     */
    public void processar(String dado) {
        // Contrato: O método não aceita dados nulos (Cláusula de Guarda)
        if (dado == null) {
            // Falha visível: informa o erro em vez de travar silenciosamente
            System.err.println("Erro: O dado fornecido para processamento não pode ser nulo.");

            // Em um sistema real, lançar uma exceção seria o ideal:
            // throw new IllegalArgumentException("O dado não pode ser nulo.");

            return; // Interrompe a execução
        }

        // Fluxo feliz (só executa se o dado for válido)
        System.out.println("Processando: " + dado.toUpperCase());
    }
}
```

Exercício 11: Implementar sinalização explícita e implícita de erros

A sinalização de erro no ValidadorCPF original é implícita. O método retorna false, mas o chamador não sabe o porquê da falha (foi nulo? tamanho errado? dígitos?).

A solução implementa uma sinalização explícita usando uma exceção customizada. O método agora lança a exceção com uma mensagem clara sobre o que deu errado.

1. Exceção Customizada

Java

```
/**
 * Exceção específica (checada) para problemas durante a validação de CPF.
 * Força o chamador a tratar o erro de validação.
 */
public class ValidacaoCPFException extends Exception {
    public ValidacaoCPFException(String mensagem) {
        super(mensagem);
    }
}
```

2. Validador Refatorado

Java

```
public class ValidadorCPF {

    private static final int TAMANHO_CPF = 11;

    /**
     * Valida um número de CPF.
     *
     * @param cpf O CPF a ser validado.
     * @return true se o CPF for válido.
     * @throws ValidacaoCPFException Se o CPF for inválido,
     * explicando o motivo da falha.
     */
    public boolean validar(String cpf) throws ValidacaoCPFException {
        // 1. Validação de Nulo (Sinalização Específica)
        if (cpf == null) {
            throw new ValidacaoCPFException("CPF não pode ser nulo.");
        }

        // Remove formatação (pontos, traços)
        String cpfLimpo = cpf.replaceAll("[^0-9]", "");

        // 2. Validação de Tamanho (Sinalização Específica)
        if (cpfLimpo.length() != TAMANHO_CPF) {
            throw new ValidacaoCPFException("CPF deve conter 11 dígitos. Tamanho recebido: " +
                cpfLimpo.length());
        }

        // (Outras lógicas de validação, ex: dígitos verificadores, viriam aqui)

        return true;
    }
}
```

Exercício 12: Tratar erros através da utilização de exceções

O método `processarPagamento` falha silenciosamente (para o sistema). Se o método for inválido, ele imprime um erro no console, mas o programa continua como se nada tivesse acontecido. O chamador (ex: a loja) não sabe que o pagamento falhou.

A solução substitui a impressão de erro pelo lançamento de uma exceção customizada. Isso para o fluxo "feliz" e força o chamador a lidar com o erro.

1. Exceção Customizada

Java

```
/**
 * Exceção (Unchecked) para sinalizar que um método de pagamento
 * não é reconhecido ou suportado pelo sistema.
 */
public class MetodoPagamentoInvalidoException extends RuntimeException {

    public MetodoPagamentoInvalidoException(String metodo) {
        super("O método de pagamento " + metodo + " não é suportado.");
    }
}
```

2. Lógica de Pagamento Refatorada

Java

```
public class Pagamento {

    /**
     * Processa um pagamento com base no método.
     *
     * @param valor O valor a ser pago.
     * @param metodo O método (ex: "cartao", "boleto").
     * @throws MetodoPagamentoInvalidoException Se o método não for reconhecido.
     */
    public void processarPagamento(double valor, String metodo) {
        if (metodo == null) {
            throw new MetodoPagamentoInvalidoException("nulo");
        }

        if (metodo.equals("cartao")) {
            System.out.println("Pagamento de R$" + valor + " realizado via cartão.");
        } else if (metodo.equals("boleto")) {
            System.out.println("Pagamento de R$" + valor + " realizado via boleto.");
        } else {
            // Lança a exceção específica em vez de imprimir um erro
            throw new MetodoPagamentoInvalidoException(metodo);
        }
    }
}
```

3. Tratamento Estruturado (Exemplo de uso)

O chamador agora pode usar try-catch para garantir um fluxo seguro.

Java

```
public class Loja {  
    public static void main(String[] args) {  
        Pagamento srvPagamento = new Pagamento();  
        double valorCompra = 150.0;  
        String metodoEscolhido = "pix"; // Método inválido  
  
        try {  
            // Tenta realizar o pagamento  
            srvPagamento.processarPagamento(valorCompra, metodoEscolhido);  
            System.out.println("Obrigado pela sua compra!");  
  
        } catch (MetodoPagamentoInvalidoException e) {  
  
            // Fluxo seguro: O erro foi capturado  
            System.err.println("Pagamento falhou. Motivo: " + e.getMessage());  
            System.out.println("Por favor, escolha 'cartao' ou 'boleto'.");  
        }  
    }  
}
```